REVISION CHEAT SHEET:

SQL SERVER FILTERING AND SORTING DATA

Interview-ready notes • examples • SQL • diagrams

When a table contains many rows, we usually do not want every row.

We often need to:

EXAMPLE / FLOW

Filter rows
+
Sort the result

The two main clauses are:

EXAMPLE / FLOW

WHERE
→ Filter rows

ORDER BY
→ Sort rows

Consider this table:

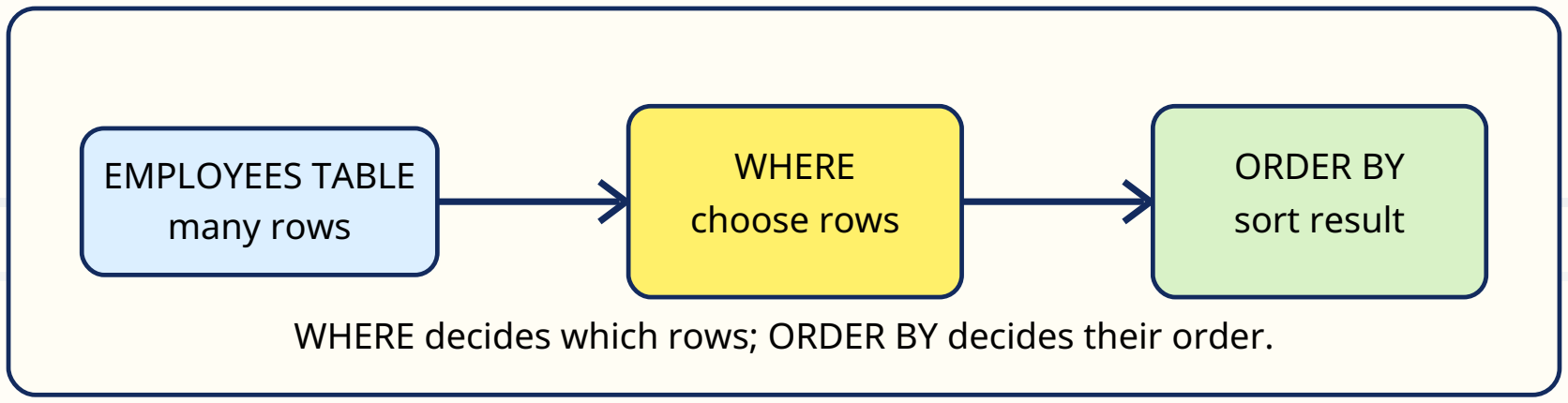
EXAMPLE / FLOW

Employees

EmployeeId	Name	Department	Salary	HireDate	City
1	John	IT	70000	2024-01-10	New York
2	Priya	HR	60000	2023-05-20	Dallas
3	David	IT	85000	2022-08-15	Boston
4	Sarah	Finance	75000	2025-02-01	NULL
5	James	IT	50000	2024-11-12	Chicago

We will use this table throughout the notes.

WHERE



Simple Definition

WHERE filters rows.

It tells SQL Server:

> Return only the rows that satisfy this condition.

Basic syntax:

```sql
SELECT column1, column2
FROM TableName
WHERE condition;
```

Example:

```sql
SELECT *
FROM Employees
WHERE Department = 'IT';
```

Result:

EXAMPLE / FLOW

```
John
David
James
```

because only those employees belong to IT.

Filtering Numbers

Numbers normally do not need quotes.

Example:

```sql
SELECT *
FROM Employees
WHERE Salary = 70000;
```

Common Comparison Operators

Operator	Meaning
=	Equal
<>	Not equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Examples:

SQL

```
SELECT *
FROM Employees
WHERE Salary > 60000;
```

SQL

```
SELECT *
FROM Employees
WHERE Salary >= 70000;
```

SQL

```
SELECT *
FROM Employees
WHERE Salary <> 70000;
```

Filtering Strings

Strings are normally written inside single quotes.

Example:

SQL

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

Another:

SQL

```
SELECT *
FROM Employees
WHERE Name = 'John';
```

Do not write:

```sql
WHERE Name = John
```

because SQL Server may interpret **John** as an identifier.

Correct:

```sql
WHERE Name = 'John'
```

String Case Sensitivity

Whether:

```
John
```

and:

```
JOHN
```

are treated as the same depends on the SQL Server **collation**.

Many SQL Server installations use case-insensitive collations, so:

```sql
WHERE Name = 'JOHN'
```

may match:

```
John
```

But this is controlled by collation settings.

Filtering Dates

Date values should be written as proper date literals.

For SQL Server, a clear format is:

```
YYYY-MM-DD
```

Example:

```sql
SELECT *
FROM Employees
WHERE HireDate = '2024-01-10';
```

Employees hired after January 1, 2024:

```sql
SELECT *
FROM Employees
WHERE HireDate >= '2024-01-01';
```

Employees hired before January 1, 2025:

```sql
SELECT *
FROM Employees
WHERE HireDate < '2025-01-01';
```

BETWEEN

Simple Definition

BETWEEN checks whether a value falls inside a range.

Syntax:

```sql
WHERE column BETWEEN value1 AND value2
```

Important rule:

> **BETWEEN** includes both boundary values.

So:

```sql
WHERE Salary BETWEEN 60000 AND 75000
```

means:

```
Salary >= 60000
AND
Salary <= 75000
```

BETWEEN with Numbers

Example:

SQL

```
SELECT *
FROM Employees
WHERE Salary BETWEEN 60000 AND 75000;
```

This includes salaries:

EXAMPLE / FLOW

```
60000
70000
75000
```

because both endpoints are included.

BETWEEN with Dates

Example:

SQL

```
SELECT *
FROM Employees
WHERE HireDate
BETWEEN '2024-01-01' AND '2024-12-31';
```

This works well when **HireDate** is a **DATE**.

Important DATETIME Rule with BETWEEN

Suppose the column is:

EXAMPLE / FLOW

```
CreatedAt DATETIME2
```

and contains:

EXAMPLE / FLOW

```
2024-12-31 15:30:00
```

This:

SQL

```
WHERE CreatedAt
BETWEEN '2024-01-01' AND '2024-12-31'
```

can be misleading because:

EXAMPLE / FLOW

```
'2024-12-31'
```

means midnight at the beginning of December 31.

A safer pattern is:

```sql
WHERE CreatedAt >= '2024-01-01'
  AND CreatedAt <  '2025-01-01';
```

This includes the entire year 2024.

This pattern is very useful for date/time filtering.

When to Use BETWEEN

Use it for clear inclusive ranges such as:

EXAMPLE / FLOW

Salary range
Age range
Quantity range
DATE columns

Example:

```sql
WHERE Age BETWEEN 18 AND 65
```

IN

Simple Definition

IN checks whether a value matches **any value in a list**.

Instead of:

```sql
WHERE Department = 'IT'
   OR Department = 'HR'
   OR Department = 'Finance'
```

you can write:

```sql
WHERE Department IN ('IT', 'HR', 'Finance')
```

Much cleaner.

IN Example

```sql
SELECT *
FROM Employees
WHERE Department IN ('IT', 'HR');
```

This returns employees whose department is either:

EXAMPLE / FLOW

IT
or
HR

IN with Numbers

SQL

```
SELECT *
FROM Employees
WHERE EmployeeId IN (1, 3, 5);
```

Equivalent to:

SQL

```
WHERE EmployeeId = 1
   OR EmployeeId = 3
   OR EmployeeId = 5
```

When to Use IN

Use when comparing one column against several possible values.

Good:

SQL

```
WHERE Status IN
(
    'Pending',
    'Approved',
    'Completed'
)
```

Instead of repeating many **OR** conditions.

NOT IN

Simple Definition

NOT IN means:

> Return rows whose value is not in this list.

Example:

SQL

```
SELECT *
FROM Employees
WHERE Department NOT IN ('HR', 'Finance');
```

This returns departments other than HR and Finance.

Important NOT IN and NULL Rule

Be careful when **NOT IN** works with a result containing **NULL**.

For example:

SQL

```
WHERE CustomerId NOT IN
(
    SELECT CustomerId
    FROM Orders
)
```

If the subquery contains a **NULL**, SQL's three-valued logic can cause surprising results.

For subquery-based exclusion, **NOT EXISTS** is often safer.

For simple fixed lists:

SQL

```
WHERE Department NOT IN ('HR', 'Finance')
```

there is no issue unless NULL handling itself matters.

Also remember:

EXAMPLE / FLOW

```
NULL NOT IN (...)
```

does not simply evaluate as true.

If you want NULL rows as well, handle them explicitly:

SQL

```
WHERE Department NOT IN ('HR', 'Finance')
   OR Department IS NULL;
```

LIKE

LIKE PATTERN MEMORY

%	_	[A-D]%
zero or more	exactly one	[ABC]%
J% / %son / %oh%	J_hn / A_	SQL Server patterns

Simple Definition

LIKE searches text using a pattern.

Syntax:

SQL

```
WHERE column LIKE 'pattern'
```

Example:

SQL

```
SELECT *
FROM Employees
WHERE Name LIKE 'J%';
```

This finds names beginning with:

EXAMPLE / FLOW

```
J
```

such as:

EXAMPLE / FLOW

```
John
James
```

SQL Server Wildcards

The two most important wildcards are:

EXAMPLE / FLOW

```
%
→ Zero or more characters

_
→ Exactly one character
```

% Wildcard

Meaning

% matches:

Zero, one, or many characters.

Example:

SQL

```
WHERE Name LIKE 'J%'
```

matches:

EXAMPLE / FLOW

J
John
James
Jennifer

because anything can appear after **J**.

Ends With

SQL

```
WHERE Name LIKE '%son'
```

matches values ending in:

EXAMPLE / FLOW

son

Examples:

EXAMPLE / FLOW

Johnson
Jackson

Contains

SQL

```
WHERE Name LIKE '%oh%'
```

matches text containing:

EXAMPLE / FLOW

oh

Example:

EXAMPLE / FLOW

John

_ Wildcard

Meaning

_ matches:

Exactly one character.

Example:

SQL

```
WHERE Name LIKE 'J_hn'
```

can match:

EXAMPLE / FLOW

```
John
```

because _ matches:

EXAMPLE / FLOW

```
o
```

But it would not match:

EXAMPLE / FLOW

```
Johan
```

because that requires more characters.

More _ Examples

SQL

```
WHERE Code LIKE 'A_'
```

matches:

EXAMPLE / FLOW

```
A1
AB
AX
```

but not:

EXAMPLE / FLOW

```
A123
```

because _ represents exactly one character.

Combining % and _

Example:

SQL

```
WHERE Name LIKE 'J_n%'
```

means:

EXAMPLE / FLOW

Starts with J

Then exactly one character

Then n

Then zero or more characters

Possible match:

EXAMPLE / FLOW

Jane...

depending on the actual characters.

SQL Server Additional Pattern Wildcards

SQL Server also supports pattern expressions such as:

EXAMPLE / FLOW

[abc]
[a-z]
[^abc]

Example:

SQL

```
WHERE Name LIKE '[JSP]%'
```

means the name begins with:

EXAMPLE / FLOW

J
S
or
P

Example:

SQL

```
WHERE Name LIKE '[A-D]%'
```

means starts with a letter from A through D.

These are useful SQL Server-specific pattern features, but **%** and **_** are the most important ones to learn first.

NOT LIKE

Simple Definition

NOT LIKE returns rows that do **not** match a pattern.

Example:

SQL

```
SELECT *
FROM Employees
WHERE Name NOT LIKE 'J%';
```

This excludes names starting with **J**.

With our example data:

EXAMPLE / FLOW

```
Priya
David
Sarah
```

would qualify.

LIKE Performance Consideration

This can often use an appropriate index more effectively:

SQL

```
WHERE Name LIKE 'John%'
```

because the pattern begins with known characters.

But:

SQL

```
WHERE Name LIKE '%John%'
```

usually makes efficient index seeking much harder because SQL Server does not know where the matching text begins.

Therefore:

EXAMPLE / FLOW

```
Prefix search
John%
→ Generally more index-friendly

Contains search
%John%
→ Often more expensive
```

For large search systems, specialized search techniques may be more appropriate.

IS NULL

Simple Definition

IS NULL finds rows where the value is NULL.

Example:

SQL

```
SELECT *
FROM Employees
WHERE City IS NULL;
```

With our sample data:

EXAMPLE / FLOW

```
Sarah
```

would be returned.

Wrong NULL Comparison

Do not write:

SQL

```
WHERE City = NULL
```

Correct:

SQL

```
WHERE City IS NULL
```

Because NULL means:

EXAMPLE / FLOW

```
Unknown / missing / not applicable
```

and does not behave like an ordinary value.

IS NOT NULL

Returns rows that contain an actual non-NULL value.

Example:

SQL

```
SELECT *
FROM Employees
WHERE City IS NOT NULL;
```

This returns employees whose city is known.

AND

Simple Definition

AND means:

All conditions must be true.

Example:

SQL

```
SELECT *
FROM Employees
WHERE Department = 'IT'
  AND Salary > 60000;
```

Both must be true:

EXAMPLE / FLOW

```
Department = IT
AND
Salary > 60000
```

From our sample data:

EXAMPLE / FLOW

```
John
David
```

qualify.

James is IT but salary is only 50000, so James does not qualify.

OR

Simple Definition

OR means:

At least one condition must be true.

Example:

SQL

```
SELECT *
FROM Employees
WHERE Department = 'HR'
   OR Department = 'Finance';
```

An employee can belong to either department.

NOT

Simple Definition

NOT reverses a condition.

Example:

SQL

```
SELECT *
FROM Employees
WHERE NOT Department = 'IT';
```

More commonly, you might write:

SQL

```
WHERE Department <> 'IT'
```

NOT is especially useful with:

EXAMPLE / FLOW

```
NOT IN
NOT LIKE
NOT BETWEEN
```

Examples:

SQL

```
WHERE Salary NOT BETWEEN 50000 AND 70000;
```

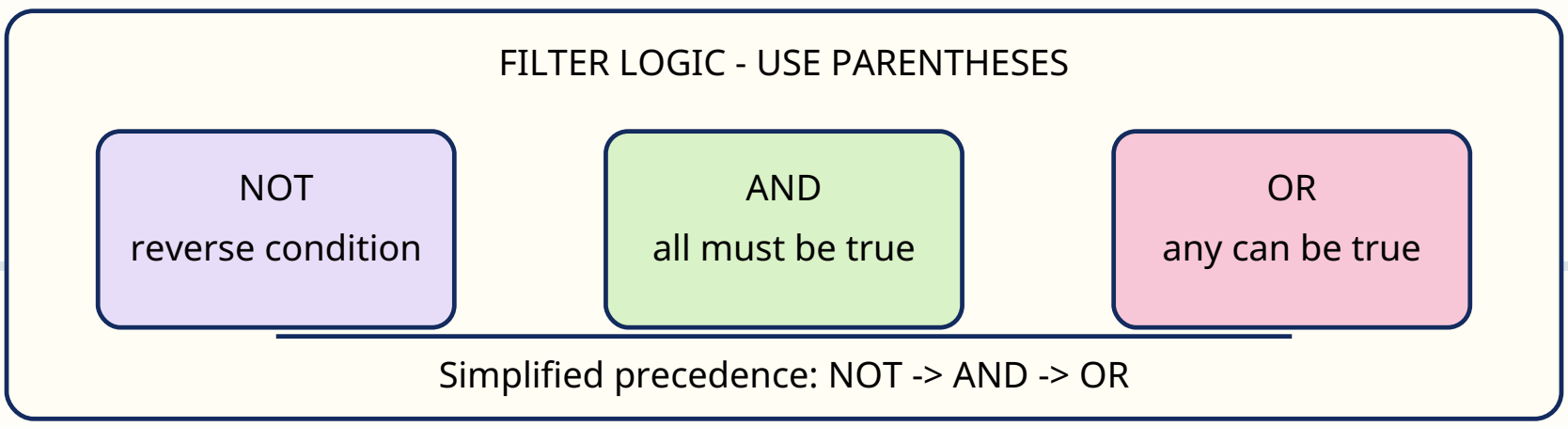
SQL

```
WHERE Department NOT IN ('HR', 'Finance');
```

SQL

```
WHERE Name NOT LIKE 'J%';
```

Combining AND and OR



This is extremely important.

Suppose you write:

```sql
SELECT *
FROM Employees
WHERE Department = 'IT'
   OR Department = 'HR'
  AND Salary > 65000;
```

This may not mean what you expect.

SQL Server evaluates:

```
AND
```

before:

```
OR
```

So the effective meaning is approximately:

```
Department = 'IT'

OR

(
    Department = 'HR'
    AND Salary > 65000
)
```

Use Parentheses for Clarity

If your actual intention is:

> Employee must be IT or HR, and salary must be over 65000

write:

```sql
SELECT *
FROM Employees
WHERE
(
    Department = 'IT'
    OR Department = 'HR'
)
AND Salary > 65000;
```

Even if you understand operator precedence, parentheses make business logic much easier to read.

AND / OR / NOT Rule

A useful simplified precedence rule is:

EXAMPLE / FLOW

```
NOT
↓
AND
↓
OR
```

But for important business queries:

Use parentheses instead of relying only on memory.

Multiple Filtering Conditions

Example:

SQL

```sql
SELECT *
FROM Employees
WHERE Department = 'IT'
  AND Salary >= 60000
  AND HireDate >= '2022-01-01';
```

Every condition must be satisfied.

Mixed Conditions

Example:

SQL

```sql
SELECT *
FROM Employees
WHERE
(
    Department = 'IT'
    OR Department = 'Finance'
)
AND Salary BETWEEN 70000 AND 90000
AND City IS NOT NULL;
```

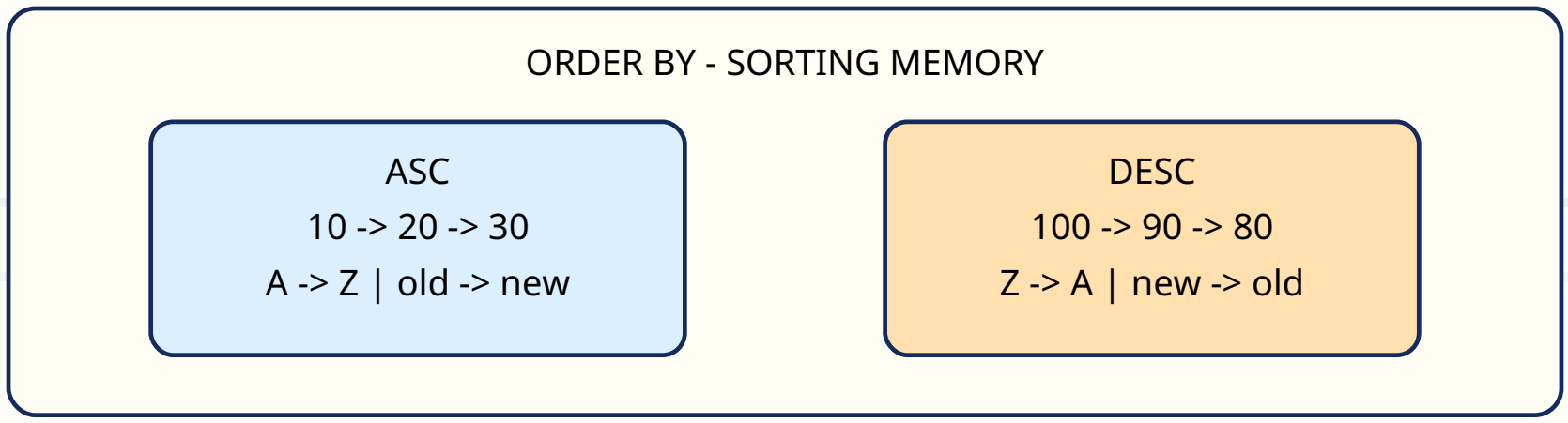
This combines:

EXAMPLE / FLOW

```
OR
AND
BETWEEN
IS NOT NULL
```

in one query.

ORDER BY



Simple Definition

ORDER BY sorts query results.

Basic syntax:

```sql
SELECT columns
FROM TableName
ORDER BY column;
```

Example:

```sql
SELECT *
FROM Employees
ORDER BY Salary;
```

By default:

```
ASC
```

is used.

ASC

ASC means:

Ascending order.

Numbers:

```
10
20
30
40
```

Dates:

EXAMPLE / FLOW

Older
→
Newer

Text:

EXAMPLE / FLOW

A
→
Z

Example:

SQL

```
SELECT *
FROM Employees
ORDER BY Salary ASC;
```

DESC

DESC means:

Descending order.

Numbers:

EXAMPLE / FLOW

100
90
80

Text:

EXAMPLE / FLOW

Z
→
A

Dates:

EXAMPLE / FLOW

Newest
→
Oldest

Example:

SQL

```
SELECT *
FROM Employees
ORDER BY Salary DESC;
```

ORDER BY Multiple Columns

Suppose:

SQL

```
SELECT *
FROM Employees
ORDER BY Department ASC,
         Salary DESC;
```

SQL Server first sorts by:

EXAMPLE / FLOW

```
Department
```

Then, employees inside the same department are sorted by:

EXAMPLE / FLOW

```
Salary DESC
```

Example:

EXAMPLE / FLOW

```
Finance
    Sarah      75000

HR
    Priya      60000

IT
    David      85000
    John       70000
    James      50000
```

WHERE and ORDER BY Together

Filtering happens conceptually before sorting the resulting rows.

Example:

SQL

```
SELECT
    EmployeeId,
    Name,
    Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC;
```

First:

EXAMPLE / FLOW

Find IT employees

Then:

EXAMPLE / FLOW

Sort those employees by salary

Result:

EXAMPLE / FLOW

David 85000
John 70000
James 50000

SQL Clause Order

When writing a basic query:

SQL

```
SELECT columns
FROM table
WHERE condition
ORDER BY column;
```

Remember:

EXAMPLE / FLOW

```
SELECT
FROM
WHERE
ORDER BY
```

You cannot normally write:

SQL

```
ORDER BY ...
WHERE ...
```

The clause order matters.

Filtering Numbers — Practical Examples

Salary greater than 70000:

SQL

```
SELECT *
FROM Employees
WHERE Salary > 70000;
```

Salary between 50000 and 80000:

SQL

```
SELECT *
FROM Employees
WHERE Salary BETWEEN 50000 AND 80000;
```

Specific employee IDs:

SQL

```
SELECT *
FROM Employees
WHERE EmployeeId IN (1, 3, 5);
```

Everything except specific IDs:

SQL

```
SELECT *
FROM Employees
WHERE EmployeeId NOT IN (1, 3, 5);
```

Filtering Strings — Practical Examples

Exact department:

SQL

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

Several departments:

SQL

```
SELECT *
FROM Employees
WHERE Department IN ('IT', 'Finance');
```

Names starting with J:

SQL

```
SELECT *
FROM Employees
WHERE Name LIKE 'J%';
```

Names not starting with J:

SQL

```
SELECT *
FROM Employees
WHERE Name NOT LIKE 'J%';
```

Names containing **a**:

```sql
SELECT *
FROM Employees
WHERE Name LIKE '%a%';
```

Filtering Dates — Practical Examples

Employees hired on a specific date:

```sql
SELECT *
FROM Employees
WHERE HireDate = '2024-01-10';
```

Employees hired after a date:

```sql
SELECT *
FROM Employees
WHERE HireDate > '2024-01-01';
```

Employees hired during 2024 when **HireDate** is **DATE**:

```sql
SELECT *
FROM Employees
WHERE HireDate BETWEEN
      '2024-01-01'
      AND
      '2024-12-31';
```

A pattern that also works safely for date/time columns:

```sql
SELECT *
FROM Employees
WHERE HireDate >= '2024-01-01'
  AND HireDate <  '2025-01-01';
```

Current Date Filtering

SQL Server provides functions such as:

```sql
GETDATE()
```

for the current SQL Server date/time.

Example:

```sql
SELECT *
FROM Employees
WHERE HireDate <= GETDATE();
```

For UTC-oriented applications, functions such as:

```sql
SYSUTCDATETIME()
```

are also useful.

Common Filtering Mistakes

Using = NULL

Wrong:

```sql
WHERE City = NULL
```

Correct:

```sql
WHERE City IS NULL
```

Forgetting Quotes Around Strings

Wrong:

```sql
WHERE Department = IT
```

Correct:

```sql
WHERE Department = 'IT'
```

Quoting Numbers Unnecessarily

Less appropriate:

```sql
WHERE EmployeeId = '10'
```

Better:

```sql
WHERE EmployeeId = 10
```

Match your literal type to your column type and avoid unnecessary implicit conversion.

Forgetting BETWEEN Is Inclusive

SQL

```
WHERE Salary BETWEEN 50000 AND 70000
```

means:

EXAMPLE / FLOW

```
Salary >= 50000
AND
Salary <= 70000
```

Using BETWEEN Carelessly with DATETIME

For all events in September 2026, prefer:

SQL

```
WHERE CreatedAt >= '2026-09-01'
  AND CreatedAt <  '2026-10-01';
```

rather than relying on:

SQL

```
BETWEEN '2026-09-01' AND '2026-09-30'
```

if time values are present.

Misunderstanding %

EXAMPLE / FLOW

```
%
```

does not mean exactly one character.

It means:

EXAMPLE / FLOW

```
Zero or more characters
```

Misunderstanding _

EXAMPLE / FLOW

```
_
```

means:

EXAMPLE / FLOW

Exactly one character

Forgetting Parentheses

Potentially confusing:

```SQL
WHERE Department = 'IT'
   OR Department = 'HR'
  AND Salary > 70000;
```

Clearer:

```SQL
WHERE
(
    Department = 'IT'
    OR Department = 'HR'
)
AND Salary > 70000;
```

Expecting ORDER BY Without Specifying It

SQL Server does **not guarantee row order** simply because rows appear in a certain order today.

If order matters:

```SQL
ORDER BY
```

must be used.

Performance Notes

Filtering performance can depend heavily on:

EXAMPLE / FLOW

Indexes
Data types
Number of rows
Predicate design
Statistics
Query structure

An indexed column may allow SQL Server to find matching rows quickly.

For example:

```SQL
WHERE EmployeeId = 100
```

can be very efficient if **EmployeeId** is indexed.

But expressions such as:

```sql
WHERE Name LIKE '%John%'
```

can be more expensive because of the leading wildcard.

Another common issue is performing functions on indexed columns:

```sql
WHERE YEAR(HireDate) = 2024
```

This may be less index-friendly than:

```sql
WHERE HireDate >= '2024-01-01'
  AND HireDate <  '2025-01-01'
```

The second form is often better for range searching.

Complete Filtering Cheat Sheet

Requirement	Syntax
Equal	`WHERE Salary = 50000`
Not equal	`WHERE Salary <> 50000`
Greater	`WHERE Salary > 50000`
Greater/equal	`WHERE Salary >= 50000`
Less	`WHERE Salary < 50000`
Range	`BETWEEN`
Several values	`IN`
Exclude values	`NOT IN`
Pattern	`LIKE`
Exclude pattern	`NOT LIKE`
Missing value	`IS NULL`
Existing value	`IS NOT NULL`
All conditions	`AND`
Any condition	`OR`
Reverse condition	`NOT`
Sort ascending	`ORDER BY ... ASC`
Sort descending	`ORDER BY ... DESC`

Wildcard Cheat Sheet

Pattern	Meaning
`%`	Zero or more characters

Pattern	Meaning
_	Exactly one character
'J%'	Starts with J
'%J'	Ends with J
'%oh%'	Contains **oh**
'J_hn'	J + exactly one char + hn
'[ABC]%'	Starts with A/B/C in SQL Server
'[A-D]%'	Starts A through D in SQL Server

AND, OR, NOT Cheat Sheet

EXAMPLE / FLOW

AND
→ Every condition must be true

EXAMPLE / FLOW

OR
→ At least one condition must be true

EXAMPLE / FLOW

NOT
→ Reverse the condition

Simplified precedence:

EXAMPLE / FLOW

NOT
↓
AND
↓
OR

Best practice:

Use parentheses whenever multiple **AND** and **OR** conditions are combined.

Final Practical Example

SQL

```
SELECT
    EmployeeId,
    Name,
    Department,
    Salary,
    HireDate
FROM Employees
WHERE
(
    Department IN ('IT', 'Finance')
)
AND Salary BETWEEN 60000 AND 90000
AND HireDate >= '2022-01-01'
AND HireDate <  '2026-01-01'
AND Name NOT LIKE 'J%'
AND City IS NOT NULL
ORDER BY
    Department ASC,
    Salary DESC;
```

This query means:

EXAMPLE / FLOW

Department
→ IT or Finance

Salary
→ 60000 through 90000

HireDate
→ Between 2022 and end of 2025

Name
→ Must not start with J

City
→ Must contain a value

Result
→ Department ascending
→ Salary descending inside department

Quick Revision

EXAMPLE / FLOW

WHERE
→ Filter

EXAMPLE / FLOW

ORDER BY
→ Sort

EXAMPLE / FLOW

BETWEEN
→ Inclusive range

EXAMPLE / FLOW

IN
→ Match any value from a list

EXAMPLE / FLOW

NOT IN
→ Exclude values from a list

EXAMPLE / FLOW

LIKE
→ Pattern search

EXAMPLE / FLOW

%
→ Zero or more characters

EXAMPLE / FLOW

_
→ Exactly one character

EXAMPLE / FLOW

IS NULL
→ Missing/unknown value

EXAMPLE / FLOW

IS NOT NULL
→ Value exists

EXAMPLE / FLOW

AND
→ All conditions

EXAMPLE / FLOW

OR
→ Any condition

EXAMPLE / FLOW

NOT
→ Opposite condition

EXAMPLE / FLOW

ASC
→ Low to high / A to Z / old to new

DESC
→ High to low / Z to A / new to old

The easiest rule to remember is:



WHERE decides which rows you want; ORDER BY decides in what order you want to see those rows.